Sejtautomata alapú oktatási rendszer

Részletes tervezés, adatbázis és REST API dokumentáció

`www` + `api` + `admin`

[Hallgató neve]

[Szak, azonosító]

Eger, 2026

Feladatkiírás

A hivatalos, aláírt feladatkiírás beszkenált változata *[ide kerüljön beszúrásként / mellékletként a tanszék utasítása szerint]*.

Szakdolgozati témaválasztó adatlap

A téma hivatalos rögzítése az intézmény *Szakdolgozati témaválasztó adatlap* űrlapján történt. A dolgozathoz csatolt, aláírással és keltezéssel ellátott változat fájlneve: **Borsos Laszlo - F4MQFM - Szakdolgozati témaválasztó adatlap.pdf** (elérési út a forrásfa szerint: `admin/szakdolgozat/`). A bevezetés fejezet *Indoklás* szakasza ennek az adatlapnak a szakmai és oktatási indoklását bővíti ki úgy, hogy a formai követelményeknek megfelelő, önálló szöveges összefoglaló is keletkezzen. Ha az adatlapon rögzített *pontos témacím* eltér a dolgozat címétől, a beadás előtt a címlapot és a hivatkozásokat szinkronizálni kell a tanszék utasítása szerint.

Nyilatkozat

[A tanszék által előírt nyilatkozat szövege (saját szellemi termék, hivatkozások, stb.).]

Köszönetnyilvánítás

[Opcionális köszönet.]

Kivonat

Ez a dolgozat a *Cellauto* nevű, három részből (`www` böngészős kliens, `api` Laravel REST szolgáltatás, `admin` React kezelőfelület) álló szoftverrendszer követelményeit, adatmodelljét, teljes API-szerződését és implementációs hátterét mutatja be. A rendszer célja a sejtautomaták interaktív oktatása: négyzetes és hexagonális rács, többféle szomszédsági szabály, felhasználói szó- és színpaletták, valamint táblaállapot-mentések REST API-n keresztül. A szöveg külön fejezetben közli a `api/docs` alapján összeállított, végpontszintű dokumentációt és az adatbázis táblák, kulcsok és idegen kulcsok részletes leírását.

Kulcsszavak: sejtautomata, REST API, Laravel, React, adatbázis-tervezés, oktatási szoftver

Minta szerinti fejezetstruktúra

Az alábbi felsorolás egy tipikus informatikai szakdolgozat tartalomjegyzékét követi; a következő oldalakon a fejezetek ehhez igazodnak:

1. Bevezetés és célkitűzés
2. Irodalomkutatás, elméleti és szakmai háttér (sejtautomaták, kapcsolódó szoftverek)
3. Követelmények és specifikáció
4. Rendszer- és adatbázis-tervezés
5. A REST API teljes dokumentációja (végpontok, modellek, hibák)
6. Az admin és a publikus kliens megvalósítása
7. Telepítés, tesztelés, biztonság, minőség
8. Összegzés és továbbfejlesztés

Tartalomjegyzék

1. Bevezetés és célkitűzés **1**

1.1. Motiváció . . . 1

1.2. Indoklás — összhang a szakdolgozati témaválasztó adatlappal . . . 1

1.2.1. Miért választottam ezt a témát? . . . 1

1.2.2. Szakmai és képzési relevancia . . . 2

1.2.3. Oktatási és felhasználói hasznosság . . . 2

1.2.4. Technológiai és piaci indoklás . . . 2

1.2.5. Személyes motiváció és tanulási cél . . . 2

1.2.6. Kockázatok és vállalás . . . 2

1.3. A fejlesztés tárgya . . . 3

1.4. Célkitűzés . . . 3

1.5. A dolgozat határa . . . 3

2. Irodalomkutatás és elméleti háttér **4**

2.1. Sejtautomata fogalma . . . 4

2.2. Szomszédságok és dimenzió . . . 4

2.3. Stephen Wolfram elemi automatái (emlékeztető) . . . 4

2.4. Conway Életjátéka . . . 5

2.5. Oktatási és demonstrációs alkalmazások . . . 5

2.6. Kapcsolódó szoftverek és eszközök (rövid áttekintés) . . . 5

2.7. Technológiai háttér (választás indoklása) . . . 5

3. Követelmények és specifikáció **7**

3.1. Funkcionális követelmények (összefoglaló) . . . 7

3.2. Nem funkcionális követelmények . . . 7

4. Rendszerarchitektúra **8**

4.1. Komponensdiagram (logikai) . . . 8

4.2. Hitelesítési folyamat . . . 8

4.3. Jogosultsági rétegek . . . 8

5. Adatbázis tervezés és felépítés **9**
5.1. Globális megközelítés . . . 9
5.2. Üzleti táblák áttekintése . . . 9
5.3. ER és UML . . . 10
5.4. Laravel infrastruktúra táblák . . . 10
5.5. Nem funkcionális szempontok (adat) . . . 10
5.6. Részletes oszlopleírások (üzleti táblák) . . . 11
5.6.1. `lists` és `words` . . . 11
5.6.2. `color_lists` és `colors` . . . 11
5.6.3. `board_save_groups` és `board_saves` . . . 11
5.7. SQL sémarészletek (illusztráció) . . . 11
5.7.1. `lists` és `words` — `CREATE TABLE` . . . 11

6. A REST API teljes dokumentációja **13**
6.1. Általános szerződés . . . 13
6.2. Végpontok összesítő táblázata . . . 13
6.3. Bejelentkezés és ping . . . 15
6.3.1. `POST /api/login` . . . 15
6.3.2. `GET /api/ping` . . . 15
6.4. Felhasználók (`api/docs/api-users.md`) . . . 15
6.4.1. `GET /api/user` és `GET /api/users` . . . 15
6.4.2. Admin műveletek . . . 15
6.5. Szólisták és szavak (`api/docs/api-lists-words.md`) . . . 15
6.6. Színpaletták (`api/docs/api-color-lists-colors.md`) . . . 15
6.7. Táblaállapot mentések . . . 16
6.7.1. Üzleti követelmény és implementáció eltérése . . . 16
6.7.2. Payload séma (ajánlott, `api-board-saves.md`) . . . 16
6.8. cURL példák (rövid) . . . 16
6.9. Táblaállapot mentések — részletes API (a `api-board-saves.md` alapján) . 16
6.9.1. Fogalmak . . . 16
6.9.2. Hitelesítés . . . 17
6.9.3. Csoport végpontok . . . 17
6.9.4. Mentés végpontok . . . 17
6.9.5. Payload séma — kötelező meta kulcsok . . . 18
6.9.6. Cellák tárolása . . . 18
6.9.7. Opcionális UI-kontextus . . . 18
6.9.8. Példa payload (JSON) . . . 18
6.9.9. Implementációs eltérések jegyzőkönyve . . . 18

7. Az admin és a publikus kliens megvalósítása **20**
7.1. Admin (React, TypeScript, Vite) 20
7.2. A `www` kliens 20

8. Telepítés, tesztelés, biztonság **21**
8.1. Telepítés 21
8.2. Tesztelés 21
8.3. Biztonság 21

9. Összegzés és továbbfejlesztés **22**
9.1. Eredmény 22
9.2. Irányok 22

A. Forráskód és dokumentáció helye **24**

1. fejezet

Bevezetés és célkitűzés

1.1. Motiváció

A diszkrét matematika és az algoritmusok oktatásában a sejtautomaták (*cellular automata*) kitűnő példák arra, hogyan vezetnek egyszerű, lokálisan értelmezett szabályok emergens, globálisan érdekes mintázatokhoz. A hallgatók számára a tananyag akkor hatékony, ha nem csupán definíciókat olvasnak, hanem *kísérleteznek*: különböző kezdeti feltételeket állítanak elő, összehasonlítják a szomszédsági módokat, és vizuálisan követik a generációkat.

1.2. Indoklás — összhang a szakdolgozati témaválasztó adatlappal

A hallgató (**Borsos László**, azonosító: **F4MQFM**) a *Szakdolgozati témaválasztó adatlap* kitöltésével formálisan vállalta a szakdolgozat témáját és annak megvalósítását. Az adatlap célja, hogy a téma *szakmai indokoltsága*, *megvalósíthatósága* és *kapcsolata a képzéshez* egy adminisztratív keretben is visszakövethető legyen; az alábbiak ezt az indoklást egészítik ki részletesen, a `admin/szakdolgozat/Borsos Laszlo - F4MQFM - Szakdolgozati témaválasztó adatlap.pdf` dokumentummal összhangban.

1.2.1. Miért választottam ezt a témát?

A sejtautomaták nem csupán matematikai érdekességek: a lokális szabályok és a globális mintázatok viszonya közvetlenül kapcsolódik az informatikus képzésben tanult *absztrakcióhoz*, *állapottér-modellezéshez* és *algoritmusok helyességének gondolatmenetéhez*. Egy interaktív, böngészőben futó alkalmazás készítése lehetővé teszi, hogy a téma ne maradjon „papíron”, hanem *működő szoftverként* jelenjen meg — ez illeszkedik a programtervező / szoftverfejlesztő kompetenciák gyakorlati kimeneteléhez.

1.2.2. Szakmai és képzési relevancia

A megvalósított rendszer teljes értékű *háromrétegű webes architektúrát* fedi le:

- **Kliens (www):** felhasználói élmény, eseményvezérelt logika, vizualizáció;
- **Kliens (admin):** típusbiztos SPA, komponensalapú felület, REST integráció;
- **Szerver (api):** erőforrás-orientált végpontok, ORM, migrációk, autentikáció és jogosultság.

Ezek együttesen a mintatanterv *webes programozás*, *adatbázisok*, *szoftvertechnológia* és *projektszerű fejlesztés* kompetenciáival igazolható összhangban vannak.

1.2.3. Oktatási és felhasználói hasznosság

A Cellauto elsődleges célcsoportja a *oktatás*: tanár demonstrációhoz és diák önálló kísérletezéshez egyaránt alkalmas. A szólisták és színpaletták API-n keresztüli kezelése lehetővé teszi, hogy a tantárgyi kontextus (pl. idegen nyelv szavai, színek jelentése) ne legyen beégetve a programba, hanem dinamikusan bővíthető legyen. A táblaállapot-mentések révén a tanóra vagy házi feladat *reprodukálható pillanatképei* visszatölthetők, ami erősíti a módszertani minőséget.

1.2.4. Technológiai és piaci indoklás

A választott technológiai stack (Laravel, Sanctum, React/TypeScript, böngészős JavaScript) iparági szempontból is releváns: a hallgató olyan eszközöket ismer meg, amelyekkel a későbbi munkahelyi környezetben is találkozhat. A REST API és az adatbázis-integritás (idegen kulcsok, egyediségi kényszerek) explicit kezelése bizonyítja, hogy a téma nem „egyszerű weblap-készítés”, hanem *szerver oldali felelősséggel* is járó fejlesztés.

1.2.5. Személyes motiváció és tanulási cél

A téma lehetőséget ad arra, hogy a hallgató a matematikai tartalom iránti érdeklődését összekapcsolja a gyakorlati fejlesztéssel: a szimuláció helyes működése, a mentések sémaszerződése és az admin felület ergonómiája egyszerre jelentenek *technikai kihívást* és *pedagógiai értéket*. A F4MQFM azonosítóval rögzített témaválasztás így nem csupán adminisztratív lépés, hanem a szakmai fejlődés egyik mérföldköve.

1.2.6. Kockázatok és vállalás

A téma vállalása azt is jelenti, hogy a dokumentáció, a tesztelés és a biztonsági szempontok (token tárolás, jogosultságok) mellett a *felhasználói minőséget* is figyelembe kell venni.

A szakdolgozat ezeket a kérdéseket külön fejezetekben tárgyalja, és a továbbfejlesztési irányokban összegzi.

1.3. A fejlesztés tárgya

A Cellauto rendszer három komponense együtt egy teljes webes ökoszisztémát ad:

- `www` — böngészőben futó sejtautomata-szimulátor oktatási célú vezérlőkkel;
- `api` — központi perzisztencia és üzleti szabályok (Laravel, JSON REST, Sanctum tokenes hitelesítés);
- `admin` — React alapú belső felület a felhasználók és a tartalomlisták (szavak, színek) kezelésére.

1.4. Célkitűzés

- A három komponens *szétválasztott* telepíthetősége és a közös API-szerződés dokumentálása.
- Felhasználóhoz kötött szó- és színpaletták, valamint táblaállapot-mentések biztosítása.
- A rendszer *egy helyen összefoglalt* szakdolgozat-szintű leírása, beleértve az adatbázis felépítését és az összes implementált REST végpontot.

1.5. A dolgozat határa

A dolgozat nem helyettesíti a forráskódot: a viselkedés részletei eltérhetnek a dokumentációtól, amíg a kódot nem frissítik. Az intézményi *feladatkiírás* és a témaválasztó adatlap *rövid, űrlapon szereplő szövegei* (ha a formai szabályzat előírja) a beadási csomag részeként külön is csatolandók; a jelen PDF a téma választását igazolja, a dolgozat pedig a megvalósítás és a dokumentáció részleteit adja.

2. fejezet

Irodalomkutatás és elméleti háttér

2.1. Sejtautomata fogalma

Sejtautomatán olyan diszkrét modellt értünk, amelyben

1. van egy cellák halmaza (rács, pl. négyzetes vagy hexagonális),
2. minden cellának van egy állapota egy véges halmazból,
3. az idő diszkrét lépésekben halad,
4. a következő állapot minden cellára *csak a szomszédok (és esetleg a cella maga) aktuális állapotától* függ, ugyanazon szabály szerint.

A szinkron frissítés azt jelenti, hogy a $t \to t{+}1$ lépésben minden cella ugyanabból a t-időbeli konfigurációból számítódik.

2.2. Szomszédságok és dimenzió

Egydimenziós automaták esetén tipikusan a cella és a bal/jobb szomszéd alkot szomszédságot; kétdimenzióban megkülönböztethető *oldalsó* (von Neumann), *csúcs* vagy *oldal+csúcs* (Moore) szomszédság, illetve hexagonális rácson más topológia. A vizsgált alkalmazás több, oktatói szempontból értelmes módot kínál, hogy a hallgató összehasonlíthassa a viselkedést.

2.3. Stephen Wolfram elemi automatái (emlékeztető)

Az egydimenziós, kétállapotú, 3 cella széles szomszédságot használó elemi automaták 256 szabálya közismert osztályozása (homogén, periodikus, kaotikus, komplex viselkedés) jól illusztrálja, hogy *minimális* változtatás a szabályban milyen drámai változást okozhat a globális dinamikában. Ez oktatási szempontból motiválja a „szabályváltás” kísérletezés

fontosságát — a Cellauto kétdimenziós és hex esetei ehhez hasonló intuíciót építenek, más geometriában.

2.4. Conway Életjátéka

A klasszikus *Game of Life* négyzetes rácson, Moore-szomszédsággal és két állapottal (élő/-halott) a legegyszerűbb példa a emergens struktúrákra (glider, oszcillátorok). A Cellauto többállapotú, több színnel és szinttel bővíti a lehetőségeket, így közelebb áll az olyan oktatási célú általánosításokhoz, ahol a cella „szintje” vagy színe tantárgyi tartalomhoz kötődik.

2.5. Oktatási és demonstrációs alkalmazások

A szimulátorok a következő pedagógiai célokat szolgálhatják:

- a *lokális szabály* → *globális minta* gondolatmenet illusztrálása;
- a *determinizmus és reprodukálhatóság* (azonos kezdeti feltétel, azonos szabály) kézzelfogható bemutatása;
- a *hipotézis–kísérlet–ellenőrzés* ciklus gyakorlása kis rácsméreteken;
- differenciálás: haladóbb hallgatók számára hexagonális topológia, kezdőknek négyzetes rács egyszerűbb szabályokkal.

2.6. Kapcsolódó szoftverek és eszközök (rövid áttekintés)

Léteznek általános CA-kutató eszközök (például a *Golly* népszerű a Life-közösségben), valamint oktatási demók böngészőben. A Cellauto eltérésként *felhasználói listákat* (szavak, színek) és *felhőbe mentett állapotokat* kapcsol a szimulátorhoz, így tantárgyi kontextusban (nyelvóra, színelmélet, projektfeladat) is használható.

2.7. Technológiai háttér (választás indoklása)

Laravel gyors REST API fejlesztés, beépített ORM, migrációk, Sanctum API token támogatás.

React + TypeScript típusbiztos admin UI, komponensalapú szerkeszthetőség.

Kliensoldali JS (www) könnyű telepítés, böngészőben futás, minimális build-lánc a publikus demóhoz.

3. fejezet

Követelmények és specifikáció

3.1. Funkcionális követelmények (összefoglaló)

Az `admin/docs/kovetelmeny-specifikacio.md` részletesen táblázatokban rögzíti az autentikációt (login, Bearer token, `GET /api/user`), a felhasználókezelést admin szerepkörrel, a szólisták és szavak teljes életciklusát (tömeges felvitel, keresés, kétlépcsős pozíciófrissítés a `UNIQUE(list_id, position)` miatt), valamint a színpaletták kezelését. A publikus kliens bejelentkezés után API-ból tölti a listákat és színeket; vendég módban korlátozottabb funkció érhető el.

3.2. Nem funkcionális követelmények

- Magyar felhasználói felület az adminban; hibák JSON üzenetből olvashatók.
- A token böngésző `localStorage`-ban történő tárolása XSS szempontból figyelendő — éles környezetben CSP és szerver oldali védelem.
- Skálázás: a listák jelenleg lapozás nélkül töltődnek; nagy adatmennyiség esetén backend oldali lapozás javasolt.

4. fejezet

Rendszerarchitektúra

4.1. Komponensdiagram (logikai)

A böngésző a `www` és `admin` klienseket futtatja; mindkettő HTTPS-en, JSON-ben kommunikál az `api` Laravel alkalmazással. Az API az adatbázishoz JDBC-hez hasonló rétegen (Eloquent ORM) fér hozzá.

4.2. Hitelesítési folyamat

1. `POST /api/login` — a kliens elküldi a `login` (email vagy felhasználónév) és `password` mezőket.
2. Sikertelen hitelesítés: `401`; felfüggesztett felhasználó: `403`.
3. Siker: válaszban `token` (Sanctum personal access token) és `user` objektum.
4. További hívások: `Authorization: Bearer <token>`, `Accept: application/json`.

4.3. Jogosultsági rétegek

- **Nyilvános:** `GET /api/ping`, `POST /api/login`.
- `auth:sanctum`: felhasználói profil, saját listák, színek, board mentések, valamint `GET /api/users` (jelen implementáció szerint minden felhasználó listázása bejelentkezve).
- `auth:sanctum` + `admin`: új felhasználó, módosítás, felfüggesztés / aktiválás (`UserController` + `AdminMiddleware`).

Megjegyzés: az `admin/docs/api-spec.md` említ `DELETE /api/users` végpontot; a jelen `routes/api.php` fájlban *nincs* regisztrálva felhasználó törlés — éles dokumentációban a tényleges útvonalak az irányadók.

5. fejezet

Adatbázis tervezés és felépítés

5.1. Globális megközelítés

A perzisztencia réteg célja a felhasználók, a felhasználóhoz kötött szó- és színpaletták, valamint a táblaállapot-mentések tárolása *referenciális integritással*. A tipikus éles környezet **InnoDB** motort és **utf8mb4** karakterkészletet használ; fejlesztéshez gyakori az SQLite. A részletes `CREATE TABLE` szövegek a `api/docs/database-schema.md` fájlban találhatók — itt a logikai szerkezet és a kulcsok összefoglalója szerepel.

5.2. Üzleti táblák áttekintése

Tábla	**Szerep és fő mezők**
`users`	Elsődleges kulcs `id`. Egyedi `email` és `username`. Mezők: `name`, `password` (hash), `role` (pl. `vendeg`, `diak`, `tanar`, `admin`), `active`, `suspended_at`, `email_verified_at`, időbélyegek. A bejelentkezés tiltása felfüggesztett vagy inaktív állapotban történik.
`lists`	Felhasználói szólisták konténere: `user_id` → `users.id`, `name`. Egy felhasználóhoz több lista (1:N).
`words`	Listaelemek: `list_id` → `lists.id`, `word` (max 255), `position` (≥ 0). **Egyediség:** `UNIQUE(list_id, word)` és `UNIQUE(list_id, position)` — ez indokolja az admin kliens kétlépcsős pozíció-frissítését cserekor.
`color_lists`	Színpaletta-listák: `user_id`, `name`.

Folytatódik a következő oldalon...

táblázat 5.1 — folytatás

Tábla	Szerep és fő mezők
`colors`	Egy paletta színei: `list_id` → `color_lists.id`, `color` (string, tipikusan hex), `position`. **Egyediség:** `UNIQUE(list_id, position)`.
`board_save_groups`	Mentési mappák: `user_id` → `users.id` (**ON DELETE CASCADE**), `name`, opcionális `position` a megjelenítési sorrendhez.
`board_saves`	Konkrét mentés: `board_save_group_id` → `board_save_groups.id` (**CASCADE**), `user_id` → `users.id` (**CASCADE**), `name`, `payload` (JSON, NOT NULL). **Egyediség:** `UNIQUE(board_save_group_id, name)`.

5.3. ER és UML

A `api/docs/database-uml.md` Mermaid és PlantUML diagramokat tartalmaz; a lényegi üzenet: a `User` központi entitás, amelyhez a listák, színes listák és mentési csoportok csatlakoznak; a `BoardSave` kettős kapcsolatot mutat (user + csoport) a gyors jogosultság-ellenőrzés és az integritás miatt.

5.4. Laravel infrastruktúra táblák

Az alkalmazás működéséhez további táblák szükségesek (nem üzleti domain): `migrations`, `sessions`, `cache`, `cache_locks`, `jobs`, `job_batches`, `failed_jobs`, `password_reset_tokens`, `personal_access_tokens`. Utóbbi tárolja a Sanctum API tokeneket; enélkül a Bearer hitelesítés nem működik.

5.5. Nem funkcionális szempontok (adat)

- A `board_saves.payload` nagy lehet — lista lekérdezéskor a jelen backend a teljes payloadot is visszaadja minden mentéshez (lásd követelmény-specifikáció az eltérésről az `api-board-saves.md` javaslatához képest).

- JSON típus: MySQL/MariaDB natív JSON; SQLite fejlesztői módban Laravel kompatibilitást biztosít.

5.6. Részletes oszlopleírások (üzleti táblák)

Az alábbi táblázatok a `database-schema.md` alapján foglalják össze az oszlopokat és kulcsokat (MySQL sémához illeszkedően).

5.6.1. `lists` és `words`

lists	`id` PK; `user_id` FK → `users`; `name` varchar(255) NOT NULL; időbélyegek.
words	`id` PK; `list_id` FK; `word` varchar(255); `position` int unsigned NOT NULL; `UNIQUE(list_id, word)`; `UNIQUE(list_id, position)`.

5.6.2. `color_lists` és `colors`

color_lists	`id` PK; `user_id` FK; `name`; időbélyegek.
colors	`id` PK; `list_id` FK → `color_lists`; `color` varchar(50); `position`; `UNIQUE(list_id, position)`.

5.6.3. `board_save_groups` és `board_saves`

board_save_groups	`id` PK; `user_id` FK CASCADE; `name`; `position` (opcionális sorrend); időbélyegek.
board_saves	`id` PK; `user_id` FK CASCADE; `board_save_group_id` FK CASCADE; `name`; `payload` JSON NOT NULL; `UNIQUE(board_save_group_id, name)`.

5.7. SQL sémarészletek (illusztráció)

A fejlesztői reprodukálhatóság érdekében az alábbiak a dokumentált `CREATE TABLE` szövegek rövidítés nélküli másolatai (a `api/docs/database-schema.md` fájlból).

5.7.1. `lists` és `words` — `CREATE TABLE`

```
CREATE TABLE `lists` (
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,
  `user_id` bigint unsigned NOT NULL,
  `name` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `fk_lists_user` (`user_id`),
  CONSTRAINT `fk_lists_user` FOREIGN KEY (`user_id`) REFERENCES `users`
     (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE `words` (
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,
  `list_id` bigint unsigned NOT NULL,
  `word` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `position` int unsigned NOT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `list_id` (`list_id`,`word`),
  UNIQUE KEY `words_list_id_position_unique` (`list_id`,`position`),
  CONSTRAINT `fk_words_list` FOREIGN KEY (`list_id`) REFERENCES `lists`
     (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Megjegyzés: az első egyedi kulcs neve a sémában `list_id`, ami összetéveszthető az oszlopnévvel; hosszabb távon beszédesebb indexnév javasolt.

6. fejezet

A REST API teljes dokumentációja

6.1. Általános szerződés

- **Alap útvonal:** minden végpont az `/api` prefix alatt érhető el (a reverse proxy konfigurációjától függően lehet relatív `/api` vagy teljes host).
- **Tartalomtípus:** kérésekben `Content-Type: application/json` (ahol van törzs); válaszban `Accept: application/json` javasolt.
- **Hitelesítés:** védett végpontokon `Authorization: Bearer <token>`.
- **Hibák:** `401` — nincs vagy érvénytelen token; `403` — nincs jogosultság (szerepkör vagy idegen erőforrás); `404` — nincs találat; `422` — validációs hiba (Laravel formátum).

6.2. Végpontok összesítő táblázata

A táblázat a `api/routes/api.php` és az `api/docs/*.md` fájlok alapján készült (2026 áprilisi állapot).

HTTP	Útvonal	Közbenső réteg / megjegyzés
GET	`/api/ping`	Nyilvános; egészségügyi válasz JSON-ban.
POST	`/api/login`	Nyilvános; body: `login`, `password`.
GET	`/api/user`	`auth:sanctum`; aktuális felhasználó.
GET	`/api/users`	`auth:sanctum`; felhasználók listája.
POST	`/api/users`	`auth:sanctum` + `admin`; új user.
PUT	`/api/users/{id}`	`auth:sanctum` + `admin`; módosítás.
POST	`/api/users/{id}/suspend`	`auth:sanctum` + `admin`.

Folytatódik...

táblázat 6.1 — folytatás

HTTP	Útvonal	Közbenső réteg / megjegyzés
POST	`/api/users/{id}/unsuspend`	`auth:sanctum` + `admin`.
GET	`/api/lists`	`auth:sanctum`; saját listák.
POST	`/api/lists`	`auth:sanctum`; body: `name`.
GET	`/api/lists/{list}`	Lista + szavak; csak tulajdonos.
PUT	`/api/lists/{list}`	Átnevezés; body: `name`.
DELETE	`/api/lists/{list}`	Lista + szavak törlése.
GET	`/api/lists/{list}/words`	Szavak `position` szerint.
POST	`/api/lists/{list}/words`	body: `word`, `position`.
PUT	`/api/lists/{list}/words/{word}`	Részleges; legalább egy mező.
DELETE	`/api/lists/{list}/words/{word}`	Szó törlése.
GET	`/api/color-lists`	Saját színes listák.
POST	`/api/color-lists`	body: `name`.
GET	`/api/color-lists/{color_list}`	Lista + színek.
PUT	`/api/color-lists/{color_list}`	body: `name`.
DELETE	`/api/color-lists/{color_list}`	Lista + színek törlése.
GET	`/api/color-lists/{color_list}/colors`	Színek rendezve.
POST	`/api/color-lists/{color_list}/colors`	body: `color`, `position`.
PUT	`/api/color-lists/{color_list}/colors/{color}`	Részleges frissítés.
DELETE	`/api/color-lists/{color_list}/colors/{color}`	Szín törlése.
GET	`/api/board-save-groups`	Saját csoportok; rendezés: `position`, `id`.
POST	`/api/board-save-groups`	Új csoport; body: `name`, opcionálisan `position`.
GET	`/api/board-save-groups/{group}`	Egy csoport.
PUT	`/api/board-save-groups/{group}`	Frissítés (implementáció: `name` kötelező).
DELETE	`/api/board-save-groups/{group}`	CASCADE mentésekre.
GET	`/api/board-save-groups/{group}/saves`	Összes mentés, **teljes payload**.
POST	`/api/board-save-groups/{group}/saves`	body: `name`, `payload` (tömb/objektum JSON).
GET	`/api/board-save-groups/{group}/saves/{save}`	Egy mentés.
PUT	`/api/board-save-groups/{group}/saves/{save}`	**Mindkettő kötelező:** `name` + `payload`.
DELETE	`/api/board-save-groups/{group}/saves/{save}`	Mentés törlése.

6.3. Bejelentkezés és ping

6.3.1. `POST /api/login`

Kérés törzs: `login` (string) — e-mail *vagy* felhasználónév; `password` (string).
Siker (200): `token` (Sanctum personal access token plaintext), `user` objektum.
Hibák: `401` `{"error":"Hibás adatok"}`; `403` felfüggesztett felhasználónál `{"error":"A felhasználó fel van függesztve"}`.

6.3.2. `GET /api/ping`

Válasz példa: `{"ok":true,"message":"API mukodik"}`.

6.4. Felhasználók (`api/docs/api-users.md`)

6.4.1. `GET /api/user` és `GET /api/users`

Bearer token szükséges. Az aktuális felhasználó illetve (implementáció szerint) az összes felhasználó rekord JSON-ban.

6.4.2. Admin műveletek

`POST /api/users` — kötelező: `name`, `email`, `username`, `password`; opcionális `role` (alap: `vendeg`).
`PUT /api/users/{id}` — módosítható: `name`, `email`, `username`, `role`, opcionális új `password`.
`POST .../suspend` és `POST .../unsuspend` — állapotváltás a dokumentáció szerint.

6.5. Szólisták és szavak (`api/docs/api-lists-words.md`)

A `words` tábla egyedisége miatt a `POST` és `PUT` műveletek ütközhetnek, ha két sor pozícióját egyszerre cserélik egy lépésben — a dokumentáció átmeneti pozíciót vagy későbbi *reorder* végpontot javasol. A `PUT /api/lists/{list}/words/{word}` hívás `422` választ ad, ha egyik frissítendő mező sincs a kérésben.

6.6. Színpaletták (`api/docs/api-color-lists-colors.md`)

A színek `list_id` és `position` egyedisége miatt ugyanaz a csere-probléma mint a szavaknál. A `color` mező hossza a sémában korlátozott (tipikusan 50 karakter).

6.7. Táblaállapot mentések

6.7.1. Üzleti követelmény és implementáció eltérése

Az `api/docs/kovetelmenyspecifikacio-tablamentesek.md` rögzíti: a `GET .../saves` lista a teljes `payload` mezőt visszaadja; a `PUT .../saves/{save}` mindkét mezőt (`name`, `payload`) kötelezően várja — ellentétben az `api-board-saves.md` korábbi „részleges frissítés” javaslatával. A szakdolgozatban a **követelmény dokumentum és a futó kód** az irányadó.

6.7.2. Payload séma (ajánlott, `api-board-saves.md`)

Kötelező meta kulcsok: `schemaVersion` (int, jelenleg 1), `board` (`square` | `hex`), `neighbors` (szomszédsági mód string), `mode` (`play` | `test`). Cellák: vagy `cells` tömb `{x,y,v}` objektumokkal, vagy `matrix` + `viewRow` + `viewCol`. Opcionális: `wordListId`, `colorListId`, `maxLevel`, `delay`, `wordMode`, `drawLevel`.
Megjegyzés: a backend a `payload`-ot strukturált JSON-ként tárolja; a részletes belső kulcsok szerinti szerveroldali séma-validáció a dokumentáció szerint *nem* kötelező végrehajtva.

6.8. cURL példák (rövid)

```
curl -X POST http://localhost:8000/api/login \
  -H "Content-Type: application/json" \
  -d '{"login":"admin@example.com","password":"secret"}'

curl http://localhost:8000/api/lists \
  -H "Authorization: Bearer TOKEN" \
  -H "Accept: application/json"
```

6.9. Táblaállapot mentések — részletes API (a `api-board-saves.m` alapján)

Ez a szakasz a mentési csoportok és mentések REST viselkedését és a `payload` ajánlott sémáját adja meg egy helyen, hogy a szakdolgozat önálló olvasásként is követhető legyen.

6.9.1. Fogalmak

Csoport: felhasználói „mappa” (pl. „Gyakorlatok”). **Mentés:** egy névvel ellátott pillanatkép a tábláról; mindig egy csoporthoz tartozik.

6.9.2. Hitelesítés

Minden végpont `auth:sanctum`. Fejlécek: `Authorization: Bearer <TOKEN>`, `Accept: application/json`, `Content-Type: application/json` (írásoknál).

6.9.3. Csoport végpontok

`GET /api/board-save-groups` A bejelentkezett felhasználó összes csoportja; mezők: `id`, `user_id`, `name`, `position`, időbélyegek; rendezés: `position` ASC, majd `id` ASC (implementáció szerint ellenőrizendő a controllerben).

`POST /api/board-save-groups` Body: `name` kötelező (max 255), `position` opcionális. `201` + létrejött objektum.

`GET /api/board-save-groups/{group}` Egy csoport; `403`, ha nem a saját.

`PUT /api/board-save-groups/{group}` A követelmény-specifikáció szerint a jelen kódban a `name` kötelező; `position` opcionális.

`DELETE /api/board-save-groups/{group}` Csoport törlése; a mentések CASCADE miatt törlődnek.

6.9.4. Mentés végpontok

`GET .../{group}/saves` A csoport összes mentése; a követelmény dokumentum szerint **teljes** `payload` minden elemnél; `403` ha nem saját csoport.

`POST .../{group}/saves` `name` kötelező, max 255, egyedi a csoporton belül; `payload` kötelező (JSON tömb/objektum); `201`.

`GET .../{group}/saves/{save}` Egy mentés teljes adatai; `403`/`404` ha nincs jogosultság vagy nincs találat.

`PUT .../{group}/saves/{save}` A **jelen implementációban mindkét mező kötelező:** `name` és `payload` (részleges frissítés nincs).

`DELETE .../{group}/saves/{save}` Mentés törlése.

6.9.5. Payload séma — kötelező meta kulcsok

Kulcs	Típus	Leírás
`schemaVersion`	int	Jelenleg `1`; változáskor növelendő.
`board`	string	`square` vagy `hex`.
`neighbors`	string	Pl. `side`, `apex`, `hex`, `life`, `life_hex` — a UI-val egyező értékek.
`mode`	string	`play` vagy `test`.

6.9.6. Cellák tárolása

A) Ritka (`cells`)**:** tömb, elemek `{ "x": int, "y": int, "v": int }`; a koordináták a belső mátrix indexelésének felelnek meg.
B) Teljes rács: `viewRow`, `viewCol`, `matrix` kétdimenziós tömb.

6.9.7. Opcionális UI-kontextus

`wordListId`, `colorListId`, `maxLevel`, `delay`, `wordMode` (`select`|`word`), `drawLevel` — ezek segítik a kliensnek a vezérlők visszaállítását betöltéskor.

6.9.8. Példa payload (JSON)

```
{
  "schemaVersion": 1,
  "board": "square",
  "neighbors": "side",
  "mode": "play",
  "maxLevel": 10,
  "delay": 0.2,
  "wordMode": "select",
  "wordListId": 2,
  "colorListId": 1,
  "cells": [
    { "x": 10, "y": 12, "v": 1 },
    { "x": 11, "y": 12, "v": 2 }
  ]
}
```

6.9.9. Implementációs eltérések jegyzőkönyve

A `kovetelmenyspecifikacio-tablamentesek.md` 8. fejezete szerint az `api-board-saves.md` korábbi javaslatai közül a PUT mentés részlegessége és a lista-válasz payload nélküli for-

mája **nincs így megvalósítva**; a szakdolgozat a követelmény-SRS és a kód szerinti viselkedést tekinti mérvadónak.

7. fejezet

Az admin és a publikus kliens megvalósítása

7.1. Admin (React, TypeScript, Vite)

Az admin SPA moduljai: bejelentkezés, felhasználók kezelése (csak `admin` szerepkörrel elérhető menü), szólisták és színpaletták. A token és a felhasználó `localStorage`-ban tárolódik (`cellauto_admin_token`, `cellauto_admin_user`). A hibák kinyerése több JSON mezőből történik (`error`, `message`, Laravel `errors`).

7.2. A `www` kliens

Vanilla JS modulok: `script.js` (rács, szabályok, generációk), `app.js` (bejelentkezés, lista- és színbetöltés, dinamikus CSS a hex színekhez), `board-save.js` (mentési csoportok és slotok UI), `api.js` (fetch wrapper). A PHP oldal routere a statikus és API-proxy útvonalakat szolgálja ki a környezettől függően.

8. fejezet

Telepítés, tesztelés, biztonság

8.1. Telepítés

`composer run setup` az API repóban: függőségek, kulcs, migráció, npm build. Fejlesztői mód: `composer run dev`. Éles: `.env`, `php artisan migrate -force`, opcionálisan queue worker.

8.2. Tesztelés

`composer run test` — PHPUnit tesztek. Az `implementacios-terv.md` további feature teszteket javasol (auth, board CRUD folyamat).

8.3. Biztonság

Sanctum tokenek; admin middleware; HTTPS; élesben CSP és XSS védelem; jelszavak hash-elve (`bcrypt`).

9. fejezet

Összegzés és továbbfejlesztés

9.1. Eredmény

A dolgozat egy összhangban lévő háromrétegű rendszert mutat be elméleti indokkal, részletes adatmodellel és teljes REST áttekintéssel.

9.2. Irányok

Form Request validáció, egységes hiba JSON, lapozás a listáknál, board lista válasz payload nélkül opcionális query-vel, reorder végpontok, bővített tesztlefedettség.

Irodalomjegyzék

[1] Wolfram, S.: *A New Kind of Science*, Wolfram Media, 2002.

[2] Laravel dokumentáció, https://laravel.com/docs

[3] React dokumentáció, https://react.dev

[4] Cellauto projekt dokumentáció, `admin/docs`, `api/docs`, `www/docs` könyvtárak.

A. függelék

Forráskód és dokumentáció helye

A `collect-sources.sh` szkript a három alkalmazás másolatát egy mappába gyűjti (`UTMUTATO.txt`). A végpontok és sémák változásakor a markdown dokumentációt célszerű a kóddal együtt frissíteni.